Advancements in Vehicle Tracking: YOLOv8 vs. YOLOv11 for Speed Estimation

Shakib Quddus
Kennesaw State University
Marrietta, Georgia
squddus@students.kennesaw.edu

***Abstract*—This paper presents a deep learning-based system for real-time vehicle speed estimation using monocular video footage. The proposed framework combines YOLO-based object detection models with the ByteTrack tracking algorithm and a geometric perspective transformation method to estimate vehicle speed in complex highway scenarios. Four YOLO variants—YOLOv11s, YOLOv11m, YOLOv8s, and YOLOv8m—are evaluated for detection accuracy, inference speed, and tracking robustness. All models are trained on the UA-DETRAC dataset and tested using high-resolution video recorded over Interstate 85 in Georgia. Evaluation results show that smaller models (YOLOv8s and YOLO11s) achieve near real-time processing speeds while maintaining high detection accuracy, making them well-suited for edge deployment. Larger models offer marginal improvements in precision but exceed real-time latency thresholds. ByteTrack enables consistent tracking in dense traffic conditions, supporting accurate frame-based speed estimation. This work demonstrates a scalable and resource-efficient solution for intelligent transportation systems and provides insights into the trade-offs between model complexity and deployability.**

***Index Terms*—component, formatting, style, styling, insert**

I. Introduction and Related Works

A. Topic Overview

Vehicle detection, tracking, and speed estimation have become essential tasks within the field of intelligent transport systems (ITS), autonomous driving, and urban traffic monitoring. These computer vision tasks enable automated systems to monitor and interpret real-time traffic conditions, ensuring road safety, efficient traffic flow, and effective law enforcement [3], [7]. Vehicle detection involves locating and classifying vehicles within a video sequence or static image, forming the initial step for subsequent tracking and speed estimation tasks. Accurate detection serves as a foundational requirement, significantly influencing the quality and reliability of downstream tasks such as tracking and velocity measurements [5], [9].

Object tracking extends detection by continuously identifying and following detected vehicles across multiple video frames, thus capturing their temporal and spatial dynamics [3], [6], [12]. Robust tracking is critical in managing occlusions, varying object scales, and complex interactions among multiple vehicles, especially in high-density traffic environments [7], [13]. Speed estimation complements detection and tracking by quantifying vehicle velocities, providing data essential for speed enforcement, accident prevention, and traffic congestion analysis [4], [14], [15]. Reliable speed measurement methods often rely on precise tracking data, accurate object localization, and advanced image-processing techniques such as optical flow or stereo-vision approaches [4], [15].

Recent advancements in deep learning, particularly the YOLO (You Only Look Once) family of object detection models, have greatly enhanced the accuracy and efficiency of these tasks. YOLO models have demonstrated superior performance due to their real-time detection capabilities, accuracy, and ability to handle complex visual scenes effectively [7], [9], [13]. Models such as the popular YOLOv8 and the recently released YOLO11 variants have emerged as prominent tools within the community, leveraging improved architectures, attention mechanisms, support for cutting edge tracking, and optimized computational efficiency [5], [9], [13]. The choice between different YOLO versions typically involves balancing detection accuracy, computational load and real-time processing capabilities, which are critical considerations for practical deployment in complex highway scenarios.

B. Problem Statement

Despite significant progress in vehicle detection, tracking and speed estimation techniques, these tasks remain challenging, particularly in complex highway environments. Highways often feature heavy traffic, fast-moving vehicles, and frequent occlusions caused by overlapping objects or varying vehicle scales [3], [7]. Additional complications arise from environmental conditions such as changes in lighting, weather variations like fog and rain, and dynamic background scenes [8], [13]. These factors negatively impact the accuracy and reliability of existing detection and tracking methodologies, causing missed detections, tracking errors, and incorrect speed estimations.

Furthermore, computational efficiency remains a critical issue for deploying these algorithms in real-world, real-time scenarios. Deep learning models, while powerful can be computationally intensive, limiting their usability in resource-constrained environments or edge devices where fast inference seeds and lower computational demands are paramount [7], [9]. Hence, there is a need to identify models that offer an optimal balance between computational efficiency and detection accuracy to enable practical implementation in challenging highway scenarios.

Fig. 1. YOLOv11 and YOLOv8 Comparison as provided by Ultralytics

C. Proposed Solution

This study addresses these challenges by systematically evaluating and comparing the performance of four state-of-the-art YOLO variants: YOLO11s, YOLO11m, YOLOv8s, and YOLOv8m. These specific YOLO models were selected due to their demonstrated capabilities in object detection accuracy, and tracking robustness. YOLOv8s and YOLOv8m are proven incumbents with many studies proving their strong performance. YOLO11s and YOLO11m on the other hand, are the newest models provided by Ultralytics which advertises the models achieving higher mAP (mean Average Precision) in the COCO dataset while using up to 22% fewer parameters [16]. The evaluation of these models focuses on their performance in a real-world complex highway scenario characterized by dense traffic, high-speed vehicle movement, and diverse environmental conditions.

The comparative analysis aims to identify the most effective model configuration for real-time highway applications by examining metrics such as detection accuracy (mAP), tracking accuracy (precision and recall) and computational efficiency (inference and processing time per frame). The findings from this analysis will offer insights into selecting optimal YOLO variants for practical deployment, balancing accuracy requirements with computational constraints, thereby enhancing the reliability and effectiveness of intelligent transportation systems.

Figure 1 shows the comparative performance of various YOLO models as described and given by Ultralytics.

D. Related Works Summary

With advancements in computer vision, and machine learning, various tracking methods have been developed to improve accuracy and efficiency. These methods can be broadly categorized into deep learning-based approaches and classic machine vision-based approaches. Deep learning-based models, leverage large datasets and computational power to provide robust tracking and detection capabilities while classical tracking and detection methods offer lightweight and interpretable solutions that can operate with fewer computational resources. This section seeks to assess the benefits and drawbacks of each of these approaches with the goal of combining methodologies to create our own solution to this complex issue and provide a real-world and complex comparison of different machine learning models to see which performs the best in a resource constrained environment.

one such study [5] performed a comparative evaluation between YOLOv5s and YOLOv7 for vehicle detection in complex and cluttered visual scenes. The results demonstrated YOLOv7's superior ability to detect small-scale vehicles especially in challenging backgrounds where traditional detectors often fail. This contribution is notable for its focus on robustness in cluttered environments—a common issue in real-world urban settings. However, the study focused exclusively on detection, omitting tracking and speed estimation which are essential for practical surveillance systems.

In another effort to benchmark YOLO's evolution, Phuong et al. [7] evaluated the performance of YOLO models ranging from version 5 to 10 using the UIT-VinaDEVES22 dataset which captures the complexity of Vietnamese traffic conditions. The study provided critical insights into model scalability and deployment feasibility. While detection accuracy improved with newer versions, the increasing model size and computational complexity posed serious limitations for edge devices and real-time deployment, particularly in resource constrained environments.

YOLOv8 has emerged as one of the most promising versions in terms of real-time applicability and detection precision. A study utilizing YOLOv8 [9] trained on an enhanced multi-class vehicle dataset achieved an impressive 99.1 accuracy rate. The model demonstrated strong generalization and detection capabilities across different vehicle categories. Despite its high performance, the model's success was heavily reliant on an optimized and curated dataset, which may not translate to performance gains in more variable or unstructured real-world data.

Building upon YOLOv8, researches have explored architectural improvements to further enhance detection accuracy in dense traffic environments. A notable approach [13] integrated the Convolutional Block Attention module (CBAM) into the YOLOv8 backbone along with soft-NMS (Non-Maximum Suppression) techniques. This configuration yielded a 2.6% improvement in detection accuracy and reduced computation overhead by 0.8 GFLOPS, reflecting a balanced trade-off between performance and efficiency. However, the added complexity in model design and longer training time can hinder fast deployment or rapid experimentation.

In addition to structural improvements, YOLO-based models have also been adapted to tackle the effects of adverse weather conditions. One innovative proposal, the "foggy env-YOLO model" [8], pre-processed input images through fog stylization and edge feature enhancement prior to feeding them into a CBAM enhanced YOLOv5 network. The model achieved a mAP of 93.6% in foggy environments and outperformed standard Faster R-CNN and YOLOv5 models in detecting larger vehicles like buses. While effective in fog, the model's reliance on a specific pre-processing pipeline may limit its generalizability across other environmental distortions such as night-time glare or rain.

While YOLO-based models dominate in terns of speed and accuracy, several researchers have explored non-YOLO alternatives for vehicle detection and tracking. One such system [6] combined SSD (Single Shot Multibox Detector) with DeepSORT and the Dlib facial landmark library to achieve reliable vehicle detection and speed estimation. The system used Euclidean distance and frame-frame displacement for speed calculation and was validated in realistic traffic scenarios. Although the system demonstrated effective multi-functionality, SSD's performance under fast motion or in densely packed scenes typically lags behind more recent YOLO versions.

Another promising approach [10] utilized a traditional computer vision pipeline based on background subtraction and morphological operations. By defining virtual detection zones and implementing adaptive thresholding and hole-filling techniques, the system achieved an accuracy of 96% for vehicle detection and counting. The primary advantage of this system lies in its simplicity and low computational demand, making it suitable for low-power environments. However, its reliance on static backgrounds makes it vulnerable to dynamic lighting, weather conditions and camera shake, which are common in real-world deployments.

Further expanding on non-YOLO approaches, [11] investigated large-scale deep learning-based object detectors trained on datasets like MS COCO and PASCAL VOC. These systems outperformed hand-crafted algorithms in recognizing small and occluded objects. Their ability to incorporate contextual information made them especially effective in dense scenes. Nonetheless, such models often require GPU acceleration, making them impractical for real-time edge deployment without considerable infrastructure support.

Vehicle speed estimation represents another critical function in ITS. Traditional methods based on stereo vision and background subtraction have evolved through the integration of machine learning and deep learning techniques. A vision-based framework proposed by [4] employed a cascade classifier for frontal vehicle detection, augmented by foreground segmentation, Kalman filtering, and the Munkres assignment algorithm for tracking. With the addition of sub-pixel matching and stereo-vision, this approach achieved superior speed tracking, although its dual-camera requirement increases deployment complexity and cost.

Timofejevs et al. [14] introduced two novel speed estimation techniques which he tested using a scale model. One was called the "sampling method" and the other was called the "skipping method" which were used with a single camera setup. These methods were validated against an infrared speed sensor which were used as ground truth. The strength of this work lies in its low-cost and single-camera design, although the accuracy was shown to degrade when vehicles moved at high speeds or when camera calibration were suboptimal.

Another technique [15] used YOLOv3 with DeepSORT, optical flow, and virtual intrusion lines to estimate vehicle speeds from monocular video. This method avoided the need for external sensors and demonstrated accuracy when compared to ground truth speed from laser guns. The hybrid design exemplified how traditional and deep learning techniques can complement on another. However, the reliance on consistent lighting and visual clarity for accurate optical flow tracking remains a limiting factor.

In terms of tracking frameworks, several deep learning-based methods have enhanced the reliability of vehicle monitoring. For example, a system proposed in [3] detected and tracked wrong-way vehicles using a combination of YOLO and DeepSORT. The solution achieved 97.53% accuracy and was successfully deployed on mobile platforms using TFLite and Core ML, demonstrating its lightweight versatility. Despite this, mobile deployment may sacrifice robustness in favor of portability.

Lastly, a vehicle tracking system proposed in [12] combined traditional blob extraction with Kalman filtering, using median filtering for background subtraction. This method was effective in stable lighting conditions and offered intuitive visual feedback using color-coded bounding boxes. Its simplicity made it attractive for real-time implementations, but the method struggled in scenes with dynamic lighting and dense object interaction.

In conclusion, the literature demonstrates that a wide range

of vehicle detection, tracking, and speed estimation models have been proposed each with its strengths and trade-offs. YOLO-based models dominate the field due to their speed and accuracy, particularly in real-time applications. However their deployment on resource-constrained devices remains a challenge, and their performance can degrade in adverse conditions without specialized modifications. Non-YOLO methods and hybrid pipelines offer compelling alternatives, particularly when combining classical computer vision with modern deep learning tools. Nonetheless, they often lack the scalability and generalization capabilities offered by end-to-end deep learning models. Continued innovation in model compression, environmental robustness, and integration of multi-modal sensing will be critical for the future development of reliable ITS solutions.

II. Project Design

The primary objective of this project is to estimate the speed of vehicles in real-world highway traffic using a deep learning-based object detection and tracking system. This solution is aimed at supporting intelligent transportation systems by enabling non-invasive, automated traffic monitoring with high accuracy and computational efficiency. To achieve this, the project integrates YOLO-based detection models with a robust tracking algorithm and a geometric perspective transformation technique to derive real-world speed measurements from monocular video footage.

At the heart of the detection pipeline are four state-of-the-art YOLO model variants: YOLO11s, YOLO11m, YOLOv8s, and YOLOv8m. These models were selected to enable a comparative analysis of detection performance across different model sizes and architectures. YOLO11, the newest iteration the YOLO family (at the time of writing this paper) has demonstrated promising improvements in multi-object tracking, segmentation, and classification tasks. Meanwhile, YOLOv8 remains one of the most widely adopted detection architectures due to its strong performance-to-speed tradeoff and ease of deployment. By comparing small (s) and medium (m) model variants from both YOLOv8 and YOLO11, the project explores the balance between inference efficiency and detection accuracy. [16]

All models were trained on the UA-DETRAC dataset, a benchmark video dataset specifically designed for vehicle detection and tracking in diverse traffic conditions. It features over 1.2 million labeled bounding boxes across over 140 thousand frames across multiple scenarios including varying lighting, weather conditions, occlusion, and dense traffic. The dataset's diversity makes it ideal for assessing the generalization capabilities of modern object detectors. Each model was trained for 50 epochs with a batch size of 16 using the AdamW optimizer. Training was conducted on an NVIDIA Quadro RTX 6000 GPU to ensure high-throughput processing and uniform training conditions. [19]

For multi-object tracking, ByteTrack was selected due to its tight integration with the Ultralytics python library, simplicity, efficiency, and superior performance on recent tracking benchmarks. ByteTrack works by associating both high-score and low-score detection boxes with existing tracks using a two-stage matching process. Unlike trackers like deepSORT that rely heavily on appearance features, ByteTrack focuses purely on detection confidence and motion-based association. This design allows it to handle crowded scenes more robustly by recovering objects that might have been missed or temporarily occluded in low-confidence detections. ByteTrack has proven particularly effective in real-world traffic video where vehicles may frequently enter and exist the field of view or are partially occluded by others. [18]

Fig. 2. Test Footage Sample Frame

After the detection and tracking steps I implemented a methodology to estimate the actual speed of detected vehicles. To do this the system employs a geometric approach based on perspective transformation. The test footage was recorded in high resolution (3840 x 2160) from a stationary camera (Canon R8 with 50 mm Lens) on a bridge above interstate 85 (I-85) in Georgia during Sunday rush hour (5:00 PM EST). This provided for a very difficult but realistic traffic scenario with many detections per frame. Because the scene was recorded from a stable reference point with precise coordinates I was able to use both satellite coordinates, and standardized road measurements to calibrate real world distances from the pixel space. Using homography, pixel displacements across consecutive frames are mapped to real-world distances, and the frame rate is used to compute instantaneous speed. This method is computationally efficient and avoids the need for specialized sensor, offering a practical solution for applications limited to monocular visual data [17]. A sample frame of the video footage used is included in Figure 2.

The models themselves were evaluated on metrics relating to detection accuracy and mAP, while the overall system was evaluated based on inference speed under varying conditions. Particular attention was given to the comparative performance of small versus medium models in terms of both detection fidelity and computational efficiency. Preliminary results suggest that while larger models like YOLO11m and YOLO8, yield higher detection accuracy, smaller models such as YOLO11s and YOLOv8s are more suitable for real-time deployment due to their significantly lower inference times.

In summary, the project design integrates modern object

detection with lightweight tracking and geometric speed estimation, forming a modular and scalable framework for real-time traffic monitoring. The use of ByteTrack, combined with efficient YOLO variants and a calibrated transformation method, provides and end-to-end pipeline that balances performance and practical deployability. Future work may involve further testing and deployment on edge platforms, or evaluating performance of these models under more difficult conditions such as night time, or inclement weather conditions.

III. Implementation and Evaluation of Approach

The implementation of this work integrates a streamlined deep learning pipeline consisting if YOLO-based object detectors and the ByteTrack tracking algorithm to estimate vehicle speed from monocular traffic footage. This section presents the empirical evaluation of the proposed system using standard performance metrics and benchmarks from both the model detection and the end-to-end vehicle monitoring perspectives. The evaluation is centered around runtime performance, detection accuracy, and the practical feasibility of deploying different model variants in real-world traffic surveillance applications.

To validate the system, four YOLO variants—YOLO11s, YOLO11m, YOLOv8s, and YOLOv8m—were tested using footage recorded from I-85 in Georgia during Sunday rush hour. The footage included dense traffic scenes (up to 25 detections per frame) with varying speeds, lighting conditions and minor occlusions. This dataset was not only used to evaluate the accuracy of the object detection, but also to benchmark the runtime performance of each model in terms of total processing time per frame. Unlike many prior works that focus exclusively on accuracy-related metrics, this study emphasizes total frame processing time as the most critical metric for real-time deployment. To simulate a relatively low power edge computing device with some hardware acceleration an Apple M1 Pro with MPS for GPU acceleration was used. A trapezoidal hot zone was used as the detection area, and area which speed calculations were done. Speed calculation was done by sampling the change in the bounding box location at regular intervals, and using the video frame rate as a temporal reference point to detect distance changed over time. Figure 4 shows what the raw tracking setup looks like with no speed or confidence thresholding, and with a light purple trapezoid showing the detection hot zone.

Each frame was passed through three core stages: (1) Preprocessing, which primarily involved image resizing and normalization; (2) Inference, which refers to the YOLO model detecting objects; and (3) Postprocessing, which includes Non-Maximum Suppression (NMS), drawing bounding boxes, and preparing output data for tracking and speed estimation. The cumulative time of these three stages determines the system's end-to-end latency per frame. Our target benchmark for what we can call "real-time" performance was 33.33 ms per frame on average due to the 30 FPS frame-rate of our video footage. Any time higher than this benchmark will result in the system falling behind real-time.

Figure 3 illustrates the clustered bar chart comparing preprocess, inference, and postprocess times across all four models. As expected smaller variants (YOLO11s and YOLOv8s) demonstrated significantly lower inference times averaging between 19-21 ms per frame with the entire pipeline taking 32.5 and 31.7 ms respectively. One thing to note between the small version of the models, YOLO11s had higher inference times but had some time savings in the postprocess cycle, but due to the slightly increased inference times, in our test case these time savings were negated. This was a similar with the medium variants (YOLO11m and YOLOv8m), except the overall pipeline time was much higher at 43.9 and 42.8 ms respectively. Again, any potential time savings from postprocessing were negated by higher inference times in the YOLO11 model. These findings support the deployment of smaller YOLO models in time-critical environments such as embedded traffic cameras or edge-based monitoring devices.

Detection performance was further evaluated using standard classification metrics including precision, recall, and mAP. These metrics were computed on the UA-DETRAC validation subset and confirmed using CometML experiment tracking. Table 1 summarizes the results across all of these models. The results indicate that all four models performed exceptionally well, with YOLOv8s achieving the highest mAP50 at 98.59%, closely followed by YOLO11m at 98.58%. In terms of the more comprehensive mAP50–95, which accounts for a broader range of IoU thresholds and is a more challenging benchmark, YOLO11m again led with 90.52%, slightly outperforming YOLOv8m at 90.26%. Interestingly, although YOLOv8s had the highest mAP50, it lagged slightly behind in mAP50–95 compared to the larger models. These results highlight that while the medium-sized models (YOLO11m and YOLOv8m) offer marginally better performance in fine-grained detection tasks, the small models remain highly competitive, especially when considering real-time processing constraints.

Model	mAP50	mAP50-95
YOLO11s	97.89%	89.46%
YOLOv8s	98.59%	89.55%
YOLO11m	98.58%	90.52%
YOLOv8m	98.41%	90.26%

TABLE I
Comparison of mAP Scores for Different YOLO Models

Figure 5-8 presents confusion matrices comparing true positive, false positive, and false negative detections for each model. These matrices further highlight the tradeoffs between precision and recall for small versus medium model variants. Larger models displayed fewer false positives but missed fast-moving, partially visible vehicles due to stricter confidence thresholds.

Model Complexity and parameter counts were also taken into account. Table 2 lists the total number of parameters per model showing that YOLO11m and YOLO8m have roughly 2-3x more parameters than their smaller counterparts. This difference directly impacts inference time, memory usage, and energy consumption, particularly on GPU-limited or embed-

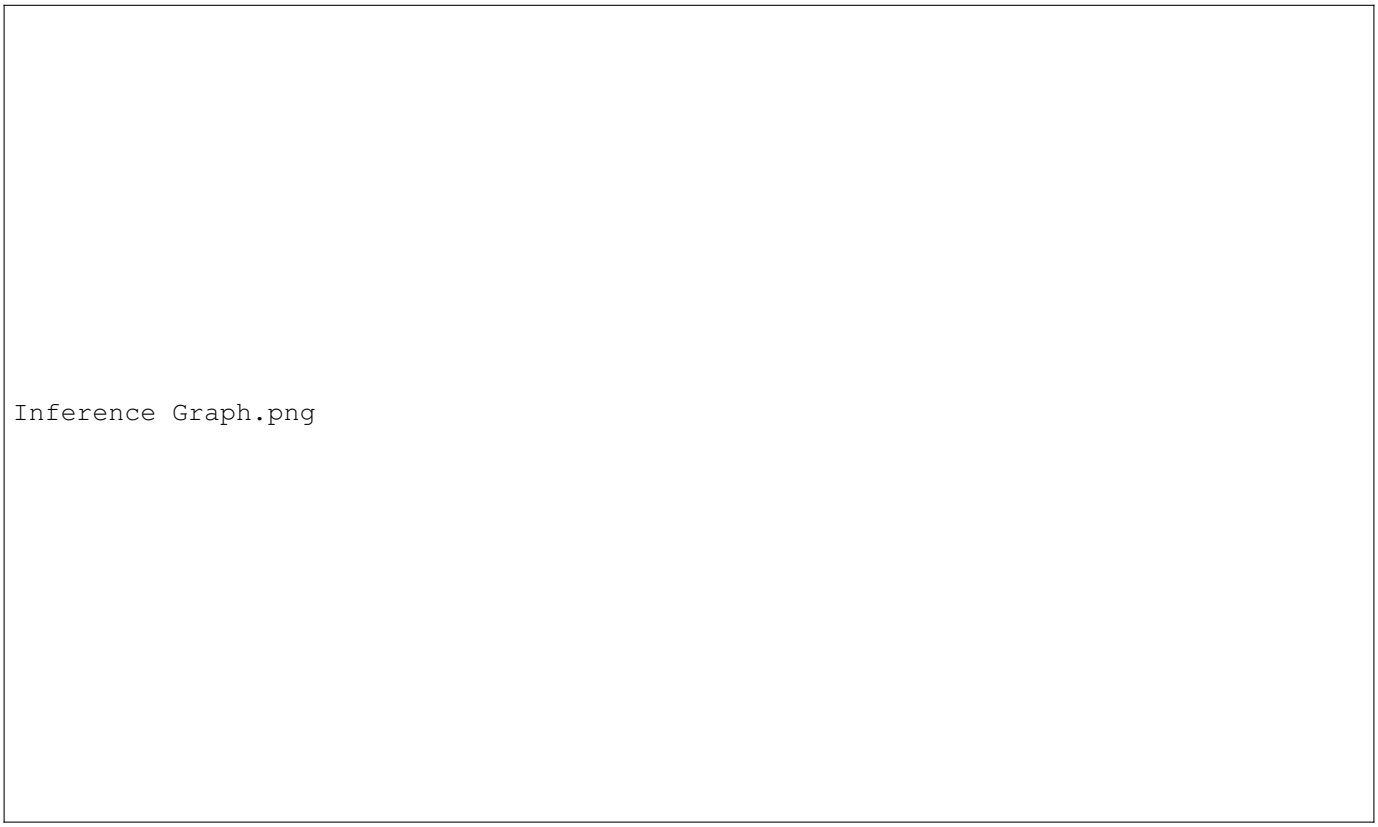

Fig. 3. YOLO Inference Time Comparison

Fig. 4. Tracking Sample Frame

ded systems. One object of note is that even though YOLO11 models had lower parameter counts than their YOLOv8 counterparts, its inference times were still higher in testing.

Model	mAP50
YOLO11s	9.4 Million
YOLOv8s	11.1 Million
YOLO11m	20.0 Million
YOLOv8m	25.9 Million

TABLE II
PARAMETER COMPARISON OF DIFFERENT YOLO MODELS

The combination of model-level and system-level evalua-

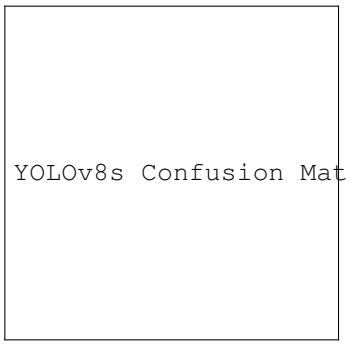

Fig. 5. YOLOv8s Confusion Matrix

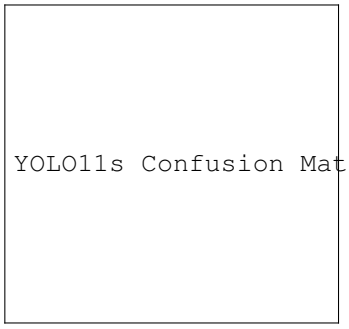

Fig. 6. YOLO11s Confusion Matrix

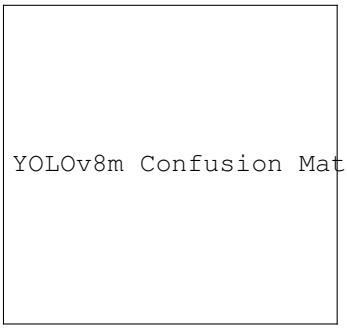

Fig. 7. YOLOv8m Confusion Matrix

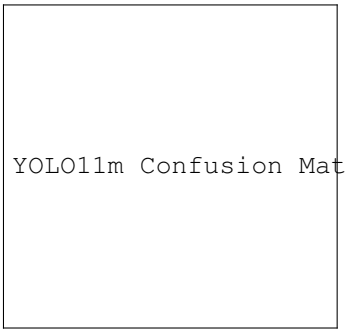

Fig. 8. YOLO11m Confusion Matrix

tion offers a comprehensive understanding of the trade-offs involved in model selection for real-time traffic monitoring. Unlike prior works that focused on either detection accuracy or tracking stability in isolation, this approach provides an end-to-end perspective, from image acquisition to speed estimation. The incorporation of ByteTrack enhances temporal consistency and enables smoother vehicle tracking over long sequences.

In summary, the experimental results show that while YOLO11m offers marginally better accuracy, its higher latency may limit real-time deployment. Conversely, YOLOv8s and YOLO11s present a viable compromise maintaining competitive accuracy, with frame processing times under our 33.33 ms threshold. ByteTrack proved effective in maintaining consistent tracking which is essential for accurate speed estimation. These findings provide key insights for selecting models based on deployment constraints, such as edge processing capability or required response time.

IV. Conclusions

This paper presented a comprehensive framework for vehicle speed estimation using monocular video input, built upon YOLO-based object detection, ByteTrack-based tracking, and a perspective transformation-based speed computation pipeline. Through a rigorous comparative analysis of four YOLO model variants—YOLO11s, YOLO11m, YOLOv8s, and YOLOv8m—this work examined critical trade-offs between detection accuracy and real-time performance under realistic traffic conditions. Using high-resolution test footage of highway traffic and a standardized evaluation dataset (UA-DETRAC), we established a reproducible experimental design to benchmark each model in terms of mAP, inference speed, post processing latency, and tracking stability.

The findings underscore that smaller models such as YOLOv8s and YOLO11s deliver competitive mAP scores (up to 98.59% mAP50) while maintaining sub-33.33 ms frame processing times, thereby satisfying real-time constraints. In contrast, medium-sized models (YOLOv8m and YOLO11m) provided modest gains in detection precision (especially mAP50-95) but at the cost of significantly increased latency—up to 44 ms per frame. Bytetrack's integration enabled reliable multi-object tracking even in dense scenes with partial occlusions, supporting robust speed estimation via geometric tracking of vehicles over time. Interestingly, despite YOLO11 models having fewer parameters than YOLOv8 counterparts, their inference latency was slightly higher, suggesting architectural inefficiencies or less mature runtime optimizations.

These results provide important guidance for selecting models based on deployment goals. For edge-based systems or embedded processors where inference time is critical, YOLOv8s or YOLO11s are well-suited options, meanwhile for cloud-based or GPU-accelerated infrastructure where latency is less of a constraint, medium models like YOLO11m offer a minor accuracy boost that may be justified in safety-critical scenarios. From a system-level perspective, the proposed framework demonstrates that even in absence of specialized hardware like LiDAR or stereo cameras, accurate vehicle speed estimation is achievable using only monocular vision and precise geometric calibration.

The study also highlights several limitations that warrant further exploration. First, while our tests included varied lighting and moderate occlusions, nighttime footage and adverse weather conditions such as rain or fog were not thoroughly evaluated. These scenarios could challenge the robustness of the models and will be prioritized in future testing. Second, the speed estimation algorithm currently assumes linear movement across the calibrated detection zone. For more complex scenes with lateral motion or sharp turns, more advanced motion modeling techniques such as optical flow integration or 3D trajectory estimation may be necessary. Lastly, the processing pipeline was tested on relatively high-performance consumer-grade hardware (NVIDIA RTX 6000 and Apple M1 Pro); future work could involve benchmarking the same models on lower-power ARM-based platforms such as the NVIDIA Jetson nano series to more realistically evaluate edge deployment feasibility.

In conclusion, this paper demonstrates that real-time vehicle detection, tracking, and speed estimation can be achieved using optimized YOLO architectures and efficient tracking pipelines. The system provides a scalable and resource-conscious foundation for intelligent traffic monitoring solutions, with flexibility for adaptation to various real-world conditions. The use of open-source tools, reproducible datasets, and modular design principles ensures that the framework is extendable, facilitating further research in ITS, smart cities, and AI-

powered infrastructure.

REFERENCES

[1] H. K. Chavda and M. Dhamecha, "Moving object tracking using PTZ camera in video surveillance system," 2017 International Conference on Energy, Communication, Data Analytics and Soft Computing (ICECDS), Chennai, India, 2017, pp. 263-266, doi: 10.1109/ICECDS.2017.8389917.

[2] B. Leibe, K. Schindler, N. Cornelis and L. Van Gool, "Coupled Object Detection and Tracking from Static Cameras and Moving Vehicles," in IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 30, no. 10, pp. 1683-1698, Oct. 2008, doi: 10.1109/TPAMI.2008.170.

[3] A. B. Siddique Mahi, F. S. Eshita and T. Helaly, "An Automated System for Wrong-Way Vehicle Detection using YOLO and DeepSORT," 2023 5th International Conference on Sustainable Technologies for Industry 5.0 (STI), Dhaka, Bangladesh, 2023, pp. 1-6, doi: 10.1109/STI59863.2023.10465068.

[4] M. Jalalat, M. Nejati and A. Majidi, "Vehicle detection and speed estimation using cascade classifier and sub-pixel stereo matching," 2016 2nd International Conference of Signal Processing and Intelligent Systems (ICSPIS), Tehran, Iran, 2016, pp. 1-5, doi: 10.1109/ICSPIS.2016.7869890.

[5] C. Gu, H. Du, X. Zhang, L. Li, Z. Yang and G. Liu, "Comparison and Application of Vehicle Target Detection Methods Based on YOLOv5s and YOLOv7," 2024 IEEE 7th International Conference on Information Systems and Computer Aided Education (ICISCAE), Dalian, China, 2024, pp. 745-749, doi: 10.1109/ICISCAE62304.2024.10761580.

[6] M. Vijayalakshmi, D. Suvitha and C. Gowtham Krishna, "Moving Vehicle Speed and Distance Estimation in Autonomous Vehicles," 2023 12th International Conference on Advanced Computing (ICoAC), Chennai, India, 2023, pp. 1-5, doi: 10.1109/ICoAC59537.2023.10249241.

[7] T. V. Phuong, C. Tran, T. Vo, K. -D. Nguyen and N. D. Vo, "Real-Time Vehicle Detection Using Surveillance Cameras: An Empirical Evaluation in Vietnamese Traffic Scenes," 2024 13th International Conference on Control, Automation and Information Sciences (ICCAIS), Ho Chi Minh City, Vietnam, 2024, pp. 1-6, doi: 10.1109/ICCAIS63750.2024.10814528.

[8] X. Wang and C. Wang, "Vehicle Multi-target Detection in Foggy Scene Based on Foggy env-YOLO Algorithm," 2022 IEEE 7th International Conference on Intelligent Transportation Engineering (ICITE), Beijing, China, 2022, pp. 451-456, doi: 10.1109/ICITE56321.2022.10101433.

[9] A. Pravallika, C. A. Kumar, E. S. Praneeth, D. Abhilash and G. S. Priya, "Efficient Vehicle Detection System Using YOLOv8 on Jetson Nano Board," 2024 IEEE International Conference on Information Technology, Electronics and Intelligent Communication Systems (ICITEICS), Bangalore, India, 2024, pp. 1-6, doi: 10.1109/ICITEICS61368.2024.10625296.

[10] N. Seenouvong, U. Watchareeruetai, C. Nuthong, K. Khongsomboon and N. Ohnishi, "A computer vision based vehicle detection and counting system," 2016 8th International Conference on Knowledge and Smart Technology (KST), Chiang Mai, Thailand, 2016, pp. 224-227, doi: 10.1109/KST.2016.7440510.

[11] A. S. Sunil Kumar and K. Mahesh, "An Investigation of Deep Neural Network based Techniques for Object Detection and Recognition Task in Computer Vision," 2023 2nd International Conference on Edge Computing and Applications (ICECAA), Namakkal, India, 2023, pp. 385-390, doi: 10.1109/ICECAA58104.2023.10212307.

[12] M. Djalalov, H. Nisar, Y. Salih and A. S. Malik, "An algorithm for vehicle detection and tracking," 2010 International Conference on Intelligent and Advanced Systems, Kuala Lumpur, Malaysia, 2010, pp. 1-5, doi: 10.1109/ICIAS.2010.5716189.

[13] X. Chen, "Vehicle Object Detection Algorithm Based on Improved YOLOv8," 2024 5th International Conference on Computer Vision, Image and Deep Learning (CVIDL), Zhuhai, China, 2024, pp. 1513-1516, doi: 10.1109/CVIDL62147.2024.10603727.

[14] J. Timofejevs, A. Potapovs and M. Gorobetz, "Algorithms for Computer Vision Based Vehicle Speed Estimation Sensor," 2022 IEEE 63th International Scientific Conference on Power and Electrical Engineering of Riga Technical University (RTUCON), Riga, Latvia, 2022, pp. 1-6, doi: 10.1109/RTUCON56726.2022.9978802.

[15] K. Sangsuwan and M. Ekpanyapong, "Video-Based Vehicle Speed Estimation Using Speed Measurement Metrics," in IEEE Access, vol. 12, pp. 4845-4858, 2024, doi: 10.1109/ACCESS.2024.3350381.

[16] G. Jocher and J. Qiu, "Ultralytics YOLO11," version 11.0.0, 2024. [Software]. Available: https://github.com/ultralytics/ultralytics.

[17] Roboflow, "Supervision: Speed Estimation Example," GitHub repository, develop branch, 2025. [Software]. Available: https://github.com/roboflow/supervision/tree/develop/examples/speed_estimation.

[18] B. Le, "An Introduction to BYTETrack: Multi-Object Tracking by Associating Every Detection Box," Datature blog, May 8, 2023. [Online]. Available: https://www.datature.io/blog/introduction-to-bytetrack-multi-object-tracking-by-associating-every-detection-box.

[19] L. Wen, D. Du, Z. Cai, et al., "UA-DETRAC: A new benchmark and protocol for multi-object detection and tracking," Computer Vision and Image Understanding, vol. 193, Art. 102907, 2020, doi: 10.1016/j.cviu.2020.102907.